F21SC- Industrial Programming

Coursework

Simple Web Browser Application

by

[F21BC] Darun Kaarthik Asokan (dka4000@hw.ac.uk)

Coursework Due Date: 23rd Oct 2025

Computer Science

School of Mathematical and Computer Science

Heriot-Watt University

Dubai Campus

1. Introduction

This report presents a detailed overview of design and the implementation of a simple web browser application, built using C# and the .NET framework. The web browser performs basics tasks like page navigation, accessing URLs, bookmarking pages and maintaining history. The various methods and classes used to achieve these functionalities are explained in the further subsections under the developer's guide.

1.1 Technical Assumptions

The application was built assuming web pages have well defined HTML structure for DOM parsing and alternatives has been developed to handle inconsistencies.

The string that handles the URL provided is coded to add **https://** prefix to avoid protocol related issues.

The app stores data corresponding to bookmarks, history and homepage as three separate text files for which the code assumes appropriate read/write permissions are provided.

UTF-8 encoding is assumed to be used by most websites to render text and HTML decoding.

Windows forms is the GUI framework used to display the front-end to the user, forms developed for this project consist of basic interface without any excessive design elements but ensure all functions work properly.

2. Requirements Checklist

Feature	Status
Sending HTTP requests and displaying the contents of the response	fully implemented
Displaying the HTTP response status code	fully implemented
Displaying the web page's title	fully implemented
Reload the current web page	fully implemented
Setting the home page	fully implemented

Adding pages to bookmarks list	fully implemented
Naming bookmarks	fully implemented
Deleting bookmarks	fully implemented
Removing bookmarks	fully implemented
Navigating to bookmarks	fully implemented
Maintaining history	fully implemented
Navigating to previous/next page in history	fully implemented
Navigating to a selected page in history	fully implemented
List five URLs from the webpage in the side panel	fully implemented
Settings (home page, bookmarks, history) persist across sessions	fully implemented
Implement database storage	Not Implemented
Add multi-user functionality	Not Implemented

3. Design Considerations

The web application is designed using an object-oriented approach. The GUI consists of three separate forms. Browser UI.cs displays the main interface to user, to perform web browsing tasks while the other two forms “BrowserEdit.cs” and “HomepageReset.cs”, supports the main interface to handle modifications of bookmarks and setting new homepage respectively.

Browser functionalities like managing history and bookmarks are categorized into separate helper classes like “BrowserHistoy.cs” and “BrowserBookmark.cs” to facilitate easy readability and maintainability.

Finally, “Program.cs” calls the mainclass to start the application.

3.1 Choice of Data Structures

Generic collection data structure is used in the code to maintain simplicity and reduce time required to process. Lists are used to store the browser history and bookmark data during runtime.

3.2 GUI Design

Further diving into the main UI built using Windows Forms (WinForms), the layout consists of buttons to navigate backwards, forwards, to load homepage directly, to reload webpage, an address bar, search button to load the URL entered in the address bar and finally a set button to set the homepage. The remaining area consists of three nested split containers, that encases a textbox inside a panel to load the html content of the page and a linklist box inside another panel which loads the top 5 HTML links in the left part and two linklistboxes encased in two panels to manage bookmarks and history respectively are placed in vertical layout in the right half. The bookmarks panel further consists of textbox with a button side-by-side to enter and add bookmarks to the browser. The historypanel also consists of a button which helps in clearing the browser history. The UI is designed to handle distortion of visual appearance during resizing of window size by docking and anchoring the various elements.

3.3 Usage of Advanced Language Constructs

Advanced constructs like asynchronous programming techniques are used to handle HTML requests. Delegates and Event handlers are also included to ensure smooth functioning of browser operations. LINQ queries and lambda expressions are used in the helper classes to perform list operations and file operations.

4. Developer Guide

The main source code of web app is stored in three files namely, BrowserUI.cs , BrowserHistory.cs and Bookmark.cs. The BrowserUI class inherits from the windows Form class to create the main front-end. It also contains code to manage the search and capturing the first 5 URLs.

BrowserUI Class
It integrates the complete functionality of the web browser in a single class.

public BrowserUI()
This constructor helps to start the GUI interface and includes Initialize browser() method to call functions to set up the http client, load the history, bookmarks from previous sessions and navigate to the saved homepage.

Methods in the class:

- NavigateToUrl(string url, bool addToHistory = true)
  Manages the search function of the web browser and returns the html content of the url. It also checks the format of each URL entered and adds the prefix https:// to reduce errors. The method handles the sending and receiving of the http request and response with error handling steps implemented.

- UpdateHistoryList()
  Refreshes the history in the listbox.

- searchButton_Click(object sender, EventArgs e)
  Navigates to the url entered on clicking the search button.

- clearHistory_Click(object sender, EventArgs e)
  clears history on clicking the clear history button and saves the result.

- backButton_Click(object sender, EventArgs e)
  Naviagates to the previous link in the history list if present on clicking the button.

- frontButton_Click(object sender, EventArgs e)
  Navigates to the previous link in the history list if present on clicking the button.

- historyListBox_DoubleClick(object sender, EventArgs e)
  Navigates to the history URL on double clicking the link from listbox.

- Form1_FormClosing(object sender, FormClosingEventArgs e)
  Saves all data on closing the web browser.

- SaveHomePage()
  Saves the homepage URL in the text file.

- LoadHomePage()
  loads the homepage link from the saved text file.

- setButton_Click(object sender, EventArgs e)
  Manages the resetting of the homepage link to the user defined link using a dialog box on clicking.

- reloadButton_Click(object sender, EventArgs e)
  Reloads the webpage on clicking.

- homePageButton_Click(object sender, EventArgs e)
  Navigates to homepage on clicking.

- PageTitle(string html)
  Captures the page title from the title tag of the html content.

- HarvestUrls(string htmlContent)
  Captures the top 5 URLs from the loaded page and displays in the listbox.

- linkListBox_Click(object sender, EventArgs e)
  Navigates to the harvested link on clicking.

- addButton_Click(object sender, EventArgs e)
  Adds the link in the search bar to the boomarks listbox on click

- UpdateBookmarksList()
  Updates the web browser with new bookmarks

- bookmarksListBox_DoubleClick(object sender, EventArgs e)
  Navigates to the selected bookmark from the listbox on double click

- deleteButton_Click(object sender, EventArgs e)
  Deletes the selected bookmark link on clicking.

- editButton_Click(object sender, EventArgs e)
  on clicking can be used to edit the name and the link of saved bookmarks.

BrowserHistory class

The BrowserHistory class manages user's browser history. It contains a list <string> as main structure to store URLs and methods to add URLs, navigate backwards and forwards, accessing clicked history link, to save the history to a text file and load it back to the program when run again.

The key logic behind how backward and forwards navigation is carried out is managed using a pointer called "CurrentIndex", which is initialized to a value of -1 at the start of the program and updates its values corresponding to method calls.

A Delegate called "NavigationHandler" is added to handle the direction of navigation.

Methods in the class:

- AddUrl(string url)
  This method takes in URL as a string, checks for null or whitespace present in the feeded URL. If condition is not satisfied it proceeds to fetch the url using uri class and extracts the url path and remove the contents after the backslash. Later the cleaned URL is compared with the contents of history list to ensure that the new entry is not a duplicate entry.

- GoBack()
  This method helps in traversing the history list backwards using CurrentIndex pointer as a reference.

- GoForward()
  This method helps in traversing the history list forwards using CurrentIndex pointer as a reference.

- Navigate(string input)
  This method uses the delegate and switch statements to call the GoBack() and GoForward() methods.

- GetHistory()
  This method is of type list and is used to store a copy of the history list for calls during runtime.

- Clear()
  This method clears the history list and sets the CurrentIndex Pointer to its initial value of -1.

- JumpToIndex(int index)
  This method is used to retrieve the URL from the specific index in the history list.

- SaveHistory() and LoadHistory()
  It is used to store the history data in text file and load back the data from the text file during runtime.

Bookmark.cs
This file manages the functions related to storing, deleting and modifying bookmarks. The file is split into two classes BrowserBookmarks and BrowserManager.

BrowserBookmarks class
It contains a constructor to store name of the bookmark and its URL and two methods:

- ToString() - Used to store the bookmark in the name|url format.

- FromString() – To retrieve the bookmark from the saved format.

BrowserManager class
It contains a list bookmarkslist <BrowserBookmarks> and methods to perform operations on bookmarks.
Methods in the class:

- AddBookmark(string name, string url) and TryBookmark(string name, string url, out BrowserBookmark added)
  AddBookmark takes in name and url as parameters and call TryBookmark method which tries to add a bookmark to the bookmarkslist after comparing the contents of the bookmarklist and if not already present in the list, it adds the bookmark to the bookmarkslist and finally the method returns true if the bookmark is added successfully else returns false. The method also has arrangements to handle errors with the name part of the bookmark.

- EditBookmark(string oldUrl, string newName, string newUrl)
  This method helps to modify the name and URL of the existing bookmarks.

- GetBookmarks()
  This method helps to populate the bookmarks listbox in the main UI.

- RemoveBookmark(string url)
  The selected bookmark can be removed from list by this method.

- SaveBookmarks() and LoadBookmarks()
  It helps to save the bookmarks in a text file and load the bookmarks back from the text file during the program execution.

Apart from these main classes and methods there are two more helper windows forms files BookmarkEdit.cs and HomepageReset.cs which aides displaying an UI through which the user can edit the bookmark and set new homepage respectively.

BookmarkEdit.cs
It contains a windows form box which is called on clicking the edit button from the main BrowserUI.

This file contains following methods:

SetBookmark(string name, string url)
which is used to set the values of the text box housing the name and url of the page to be bookmarked.

okButton_Click(object sender, EventArgs e)
On clicking Ok the methods validates and saves he newly entered information.

HomepageReset.cs
Displays the user with a windows form with an textbox to set the new homepage link.

This file contains following methods:

string HomePageUrlSetter
This method fetches the previous url entered and sets the new url entered by the user in the textbox.

Libraries Used

For building the web browser the following libraries were used:
System.Text.RegularExpressions , System.HtmlAgilityPack, System.Collections.Generic, System.IO, System.Linq, System.Windows.Forms, System.Net.Http

5. Reflections on Programming Language and Implementations

This course has taught me a lot about C#, the project work has made me research further about the tasks that can be completed using C#. For this particular project C#'s .NET framework and the System.Net.Http library has been immensely useful to process the html request and response. Further inbuilt features such as automatic garbage collection ensures constant performance throughout the runtime of the web browser. C#'s windows forms library, which is easy to use due to its drag and drop feature was also greatly during designing the entire front-end of the project. The project in spite of being developed meticulously has few shortfalls such as storing of files locally in text files and lack of multiuser features, which can be addressed in the future versions of the program.

6. Conclusion

The web browser application which I developed has, classes and methods designed based on Object Oriented programming concepts and try-catch loops to capture most of the errors encountered during run-time to prevent the unexpected crashing of the program during run-time. The program also includes a simple clean UI with asynchronous programming to ensure smooth flow of the UI. However, implementation of multiuser functionality and storing history, bookmarks and homepage data in database would have been better.

7. References

https://learn.microsoft.com/en-us/dotnet/desktop/winforms/
https://learn.microsoft.com/en-us/dotnet/api/system.net.http.httpclient?view=net-9.0
https://html-agility-pack.net/documentation
https://www.geeksforgeeks.org/c-sharp/introduction-to-c-sharp-windows-forms-applications/